

Module 2: POS Tagging & Hidden Markov Models

Penn Treebank Tagset, HMM 5-Tuple, Forward Algorithm, Viterbi Dynamic Trellis & CRFs

Module II: Language Models, POS Tagging & Parsing — Complete 12-Topic Tracker

Topic 13: N-Gram Language Models (Unigram, Bigram & Trigram Probabilities)

Topic 15: Rule-Based Part-of-Speech Tagging (Morphological Rules)

Topic 17: Context-Free Grammars (CFG Formal Definition & Parse Trees)

Topic 19: Probabilistic Parsing (PCFG, CNF & CKY Algorithm) [UPLOADED PYQ]

Topic 21: Precision Metric Formulation in NLP Evaluation

Topic 23: F1-Score Harmonic Mean Formulation

Topic 14: Smoothing & Perplexity (Laplace Add-1 & Kneser-Ney Backoff)

Topic 16: HMM POS Tagging & The Viterbi Algorithm [UPLOADED PYQ]

Topic 18: Recursive Descent Parsing (Top-Down Backtracking Search)

Topic 20: Dependency Parsing (Typed Dependency Trees & Arc-Standard)

Topic 22: Recall Metric Formulation in Information Extraction

Topic 24: Parsing Evaluation Metrics (UAS & LAS Attachment Scores)

Topic 13 & 14: N-Gram Models, Laplace Smoothing & Perplexity [UPLOADED PYQ]

1. Bigram Maximum Likelihood Estimation (MLE):

$$P(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i)}{\text{Count}(w_{i-1})}$$

2. Laplace (Add-1) Smoothing Formula [UPLOADED PYQ]:

$$P_{\text{Laplace}}(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i) + 1}{\text{Count}(w_{i-1}) + |V|}$$

Where $|V|$ is the total vocabulary size. Guarantees zero-probability events never crash sentence likelihoods!

3. Model Perplexity Metric (PP):

$$PP(W) = P(w_1, w_2, \dots, w_N)^{-\frac{1}{N}} = \sqrt[N]{\frac{1}{P(w_1, w_2, \dots, w_N)}}$$

Interpretation: The average branching factor of the language model (Lower Perplexity $\implies$ Higher Predictive Accuracy).

[UPLOADED PYQ] Solved Problem: Bigram Probability with Laplace Smoothing

Given corpus counts: $\text{Count}(\text{"want"}, \text{"to"}) = 600$, $\text{Count}(\text{"want"}) = 1200$, Vocabulary $|V| = 5000$.

1. MLE Probability: $P_{\text{MLE}}(\text{"to"} | \text{"want"}) = \frac{600}{1200} = \mathbf{0.500}$.

2. Laplace Smoothed Probability: $P_{\text{Laplace}}(\text{"to"} | \text{"want"}) = \frac{600+1}{1200+5000} = \frac{601}{6200} \approx \mathbf{0.0969}$.

Topic 15 & 16: HMM POS Tagging & The Viterbi Algorithm [UPLOADED PYQ]

A **Hidden Markov Model (HMM)** models Part-of-Speech tagging as finding the most probable hidden sequence of POS tags $T = \langle t_1, \dots, t_n \rangle$ given observed words $W = \langle w_1, \dots, w_n \rangle$:

$$\hat{T} = \arg \max_T P(T | W) = \arg \max_T \prod_{i=1}^n P(t_i | t_{i-1}) \cdot P(w_i | t_i)$$

The Viterbi Dynamic Programming Algorithm [UPLOADED PYQ]

$$V_t(j) = \max_{i=1}^N [V_{t-1}(i) \cdot a_{ij}] \cdot b_j(o_t)$$

$$\text{Backpointer: } B_t(j) = \arg \max_{i=1}^N [V_{t-1}(i) \cdot a_{ij}]$$

Where $a_{ij} = P(t_j | t_i)$ is Tag Transition probability, and $b_j(o_t) = P(w_t | t_j)$ is Word Emission probability.

Topic 17 – 20: Syntactic Parsing (CFG, PCFG, CKY & Dependency Parsing) [UPLOADED PYQ]

Parsing Paradigm	Grammar & Structural Formulation	Key Advantages & Tradeoffs
1. Context-Free Grammar (CFG)	$G = (V, \Sigma, R, S)$ where productions have form $A \rightarrow \alpha$ ($A \in V$). Recursive descent expands rules top-down.	Precise; vulnerable to left-recursion infinite loops and combinatorial ambiguity explosions.
2. Probabilistic CFG (PCFG) [UPLOADED PYQ]	Assigns probability to each rule: $P(A \rightarrow \alpha)$, where $\sum_{\alpha} P(A \rightarrow \alpha) = 1$. Parse probability is product of rule probabilities.	Ranks multiple ambiguous parse trees and selects the mathematically most probable syntactic structure!
3. CKY Dynamic Programming [UPLOADED PYQ]	Requires grammar in Chomsky Normal Form (CNF) : $A \rightarrow BC$ or $A \rightarrow a$. Fills triangular table $O(n^3 \cdot R)$.	Guaranteed polynomial $O(n^3)$ parsing time for all ambiguous sentences!
4. Dependency Parsing	Connects words directly with labeled binary grammatical dependency links (e.g., nsubj, dobj, amod).	Directly captures predicate-argument relationships; excellent for free-word-order languages.

Topic 21 – 24: Parsing & Classification Evaluation Metrics

1. Classification / Parsing Core Metrics:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1\text{-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **Unlabeled Attachment Score (UAS)**: Percentage of words assigned the correct syntactic head governor. - **Labeled Attachment Score (LAS)**: Percentage of words assigned both the correct head governor AND the correct dependency relation label.



M2 Active Recall & Exam Questions [UPLOADED PYQ]

Q1. [UPLOADED PYQ] Explain the working of HMM-based POS Tagging. Define Transition and Emission Probabilities. (10 Marks)

HMM treats POS tags as hidden Markov states generating observable surface words:

1. **Transition Probability** $P(t_i | t_{i-1})$: The probability of transitioning from tag t_{i-1} to tag t_i : $P(t_i | t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}$.
2. **Emission Probability** $P(w_i | t_i)$: The probability that hidden tag t_i emits the observable word w_i : $P(w_i | t_i) = \frac{C(t_i, w_i)}{C(t_i)}$.
3. **Viterbi Algorithm**: Computes the global optimal sequence of tags in $O(T \cdot N^2)$ time using dynamic programming lattice trellis!